COMP2870 Theoretical Foundations: Calculus II

Thomas Ranner

23 January 2026

Table of contents

1 Introduction

1.1 Contents of this submodule

This part of the module will deal with numerical algorithms that involve derivatives and integrals. We will approach these problems using a combination of theoretical ideas and practical solutions, thinking through the lens of real-world applications. As a consequence, to succeed in linear algebra, you will do some programming (using Python) and some pen-and-paper theoretical work, too.

1.1.1 Topics

We will have 6 double lectures, 2 tutorials, and 3 labs. We break up the topics as follows:

Lectures

1. Introduction to ordinary differential equations and Euler's method (week 9, L1)
2. The midpoint method and systems of differential equations (week 9, L2)
3. Introduction to optimisation and partial derivatives (week 10, L1)
4. Nonlinear solvers (week 10, L2)
5. Gradient flow (week 11, L1)
6. Using gradient flow to train a neural network (week 11, L2)

Labs

1. Ordinary differential equations (Week 9)
2. Nonlinear solvers (Week 10)
3. Using gradient flow to train a neural network (Week 11)

Tutorials

1. Ordinary differential equations (Week 11)
2. Optimisation (Week 12)

1.1.2 Learning outcomes

Candidates should be able to:

TODO

1.2 Textbooks and other resources

There are many textbooks and other external resources which could help your learning:

TODO

1.3 Programming

This section of the notes links theoretical material with practical applications. We will have weekly lab sessions where you will see how the methods mentioned in the lecture notes work when implemented. More instructions on how we will do this will be covered in the lab sessions themselves.

An important theoretical aspect of translating the algorithms to computer implementations is the use of floating-point numbers. You will have already met floating-point numbers in your first year studies where you met how computer store numbers. You will already know that computer cannot store every possible real number exactly. The inexactness of floating-point numbers has real consequences when performing computations.

Exploring and understanding the material in () is really helpful for understanding many of the choices that we make when forming methods in linear algebra.

We will make extensive use of `Python` and `numpy` during this section of the module. It may help you to revise some of your notes from last year on these topics.

1.4 The big problems for this part of the module

1.4.1 Differential equations

1.4.2 Nonlinear solvers

1.4.3 Optimisation

1.5 Comments on these notes

There are two versions of these notes available: as an online website and as a pdf. I expect minor adjustments to be made to both throughout the progress of delivering the materials.

You can always find the latest versions at:

TODO

Please either email me (T.Ranner@leeds.ac.uk) or use the github repository to report any corrections.

`This version of the notes is a90fde1+dirty.`

2 Introduction to dynamic problems

There are many models and problems where time plays an important role.

Examples TODO

Often these models use the language of derivatives and calculus to express the underlying processes. We will spend the first two lectures of this part of the module covering how we can solve this type of dynamic problem using numerical methods.

As a computer scientist, this will help you in your future studies by introducing:

- important methods for solving interesting problems often found in science and engineering, robotics and games design;
- the idea of splitting up a continuous problem into a finite number of steps which makes a tractable problem for a computer to solve;
- measuring errors and efficiency of approaches with a view to balancing between them.

2.1 Rates of change

We will start with a recap of material that you learnt last year to define the objects we want to discuss. We will do this in a way that's driven by our applications and methods which will follow.

Suppose we know that a person is walking at a *constant* speed of 3 meters per second (m/s). What does that really mean? How far will they travel in:

- 0.1 seconds?
- 0.01 seconds?
- 0.001 seconds?
- 10^{-6} seconds?

What does this tell us about speed?

We have that

$$\text{Distance travelled} = \text{speed} \times \text{time},$$

or equivalently:

$$\text{speed} = \frac{\text{distance travelled}}{\text{time}}.$$

What if the object's speed was not constant? For example, we could define the speed

$$S(t) = -(t-1)(t+0.5). \tag{2.1}$$

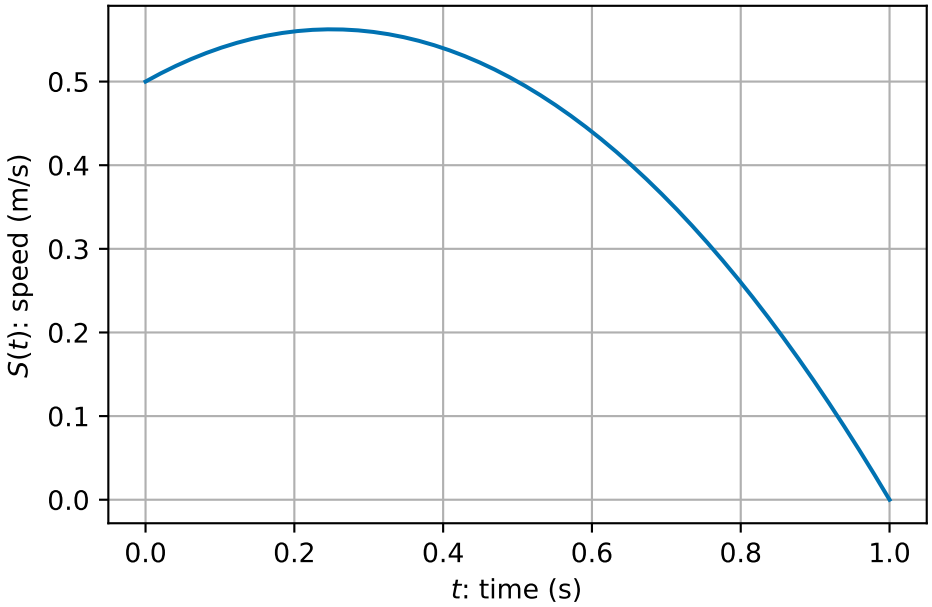


Figure 2.1: A sample object speed over time

How far would the object travel in one second now?

We could consider each tenth of a second separately and estimate the distance covered at each tenth of a second (assuming the $S(t)$ is approximately constant in each interval):

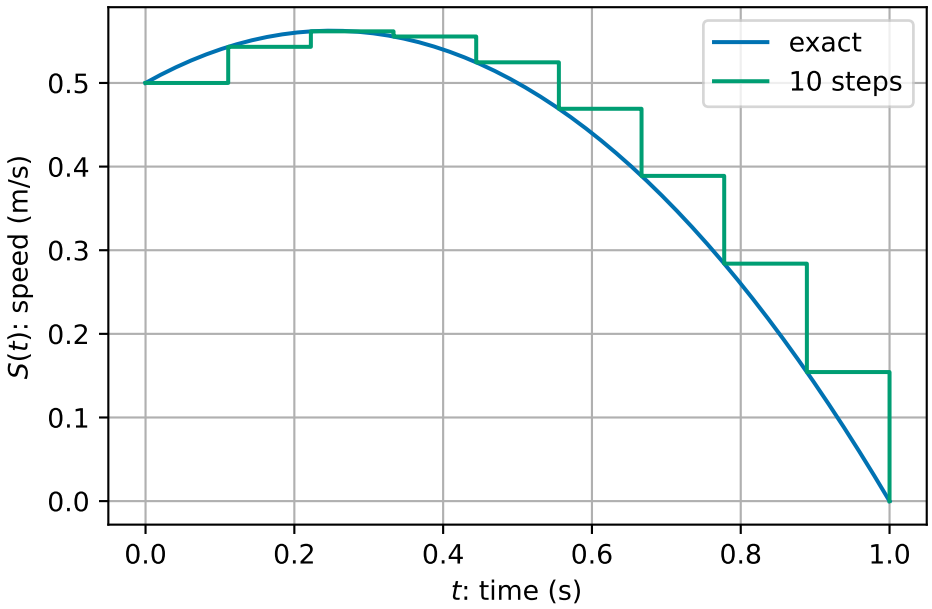


Figure 2.2: A sample object speed over time approximated over 10 steps

```
D, t = 0.0, 0.0

for _i in range(10):
    D = D + 0.1 * S(t)
    t = t + 0.1

print(D, t)
```

```
0.44000000000000006 0.9999999999999999
```

We could consider the same calculation of hundredths of seconds assuming that the speed is approximately constant in each interval:

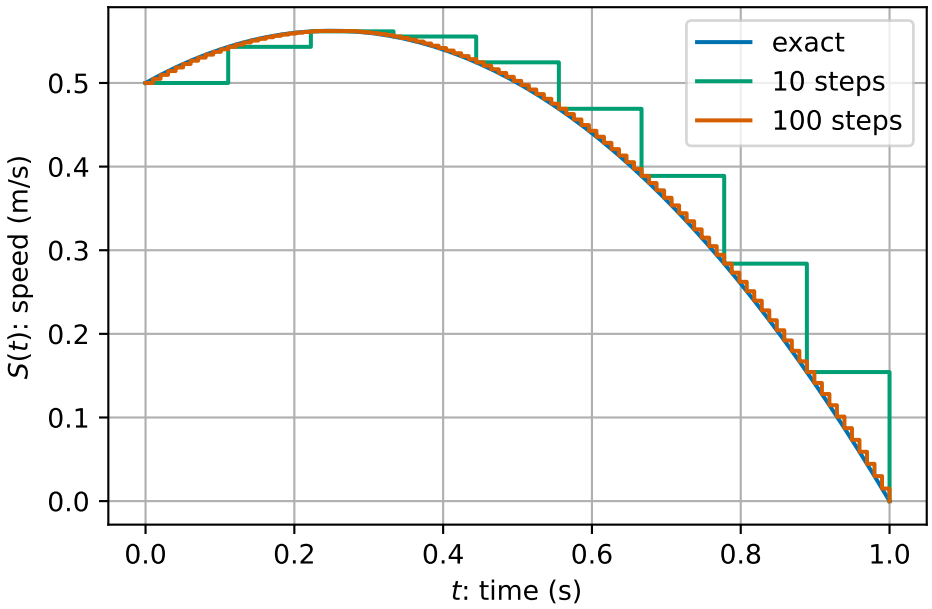


Figure 2.3: A sample object speed over time approximated over 100 steps

```
D, t = 0.0, 0.0

for _i in range(100):
    D = D + 0.01 * S(t)
    t = t + 0.01

print(D, t)
```

```
0.41914999999999963 1.0000000000000007
```

What about if we did a thousand steps

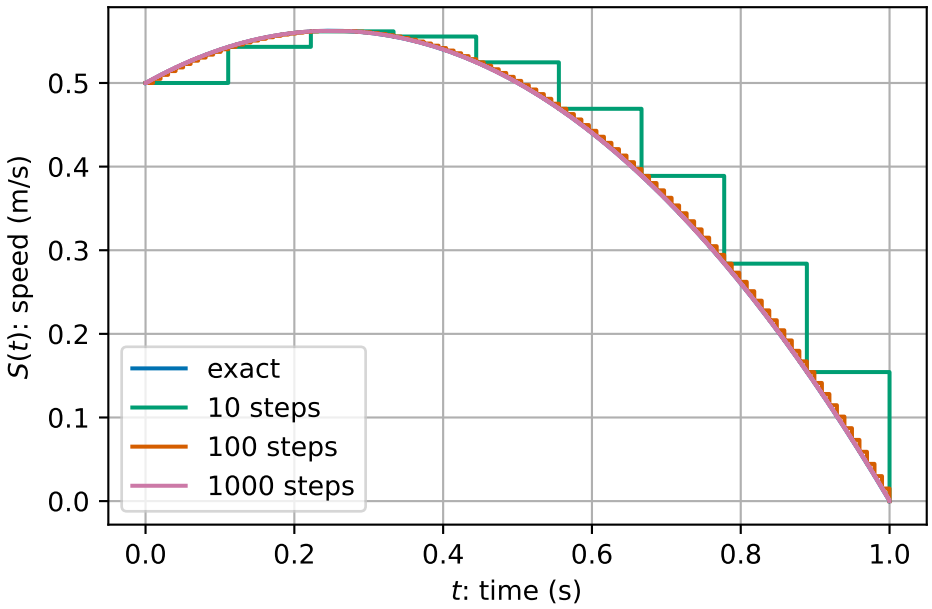


Figure 2.4: A sample object speed over time approximated over 1000 steps

```
D, t = 0.0, 0.0

for _i in range(1000):
    D = D + 0.001 * S(t)
    t = t + 0.001

print(D, t)
```

```
0.41691649999999947 1.0000000000000007
```

In summary, we get

	intervals	increment size, h	total distance
0	1	1.00000	0.500000
1	10	0.10000	0.440000
2	100	0.01000	0.419150
3	1000	0.00100	0.416916
4	10000	0.00010	0.416692
5	100000	0.00001	0.416669

We appear to be converging to a limit as $h \to 0$...

We might express this mathematically as:

$$S(t) = \lim_{h \to 0} \frac{D(t+h) - D(t)}{h},$$

or that *instant* speed is the limit of average speed over smaller and smaller intervals.

Formally, we say that speed, $S(t)$, is the **derivative** of distance, $D(t)$, with respect to time and write $S(t) = D'(t)$.

2.2 Geometric picture

We can plot the expression for the speed, $S(t)$, from (2.1) and it's integral from 0 up to time t, which we call the distance. $D(t)$.

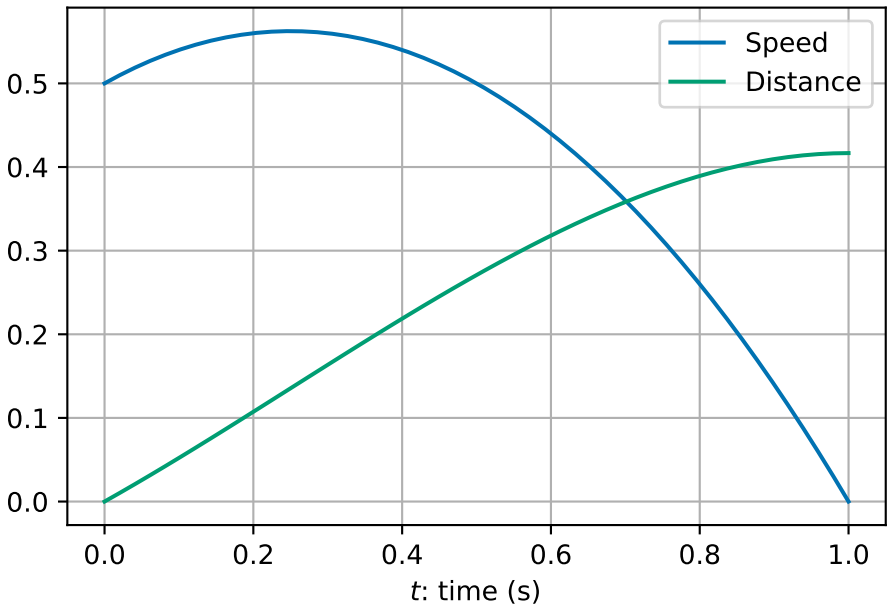


Figure 2.5: Distance and speed examples as a function of time

We see that the **gradient** (slope) of the green curve is the **value** of the blue curve.

- As the blue curve increases in value, the green curve increases in steepness.
- When the blue curve starts to decrease in value, the green curve decreases in steepness.
- By the time the blue curve has a value of zero the green curve is flat.

This gives us an alternative definition of derivative:

> The function $D'(t)$ is the function whose value is equal to the slope (or gradient) of $D(t)$ at every value of t.

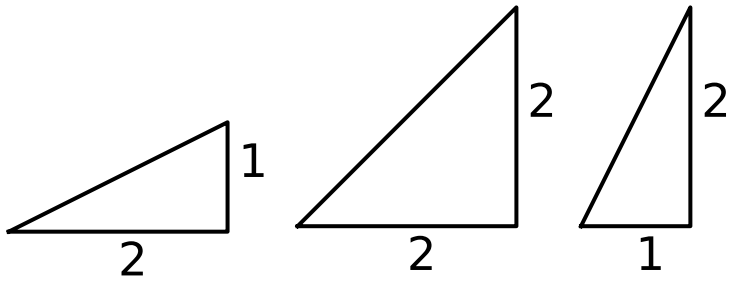


Figure 2.6: Examples of different sloped straight lines.

For a straight line, we can use the formula for a straight line to find the gradient. The equation of a straight line with slope m is given by

$$f(t) = mt + c.$$

For a curve (i.e. non-straight line), we can approximate the slope with a straight line approximation ("chord"):

$$\frac{f(t+h) - f(t)}{h}.$$

We can get a better approximation by taking a smaller value of h…. This leads us back to the definition of derivative (Definition 2.1).

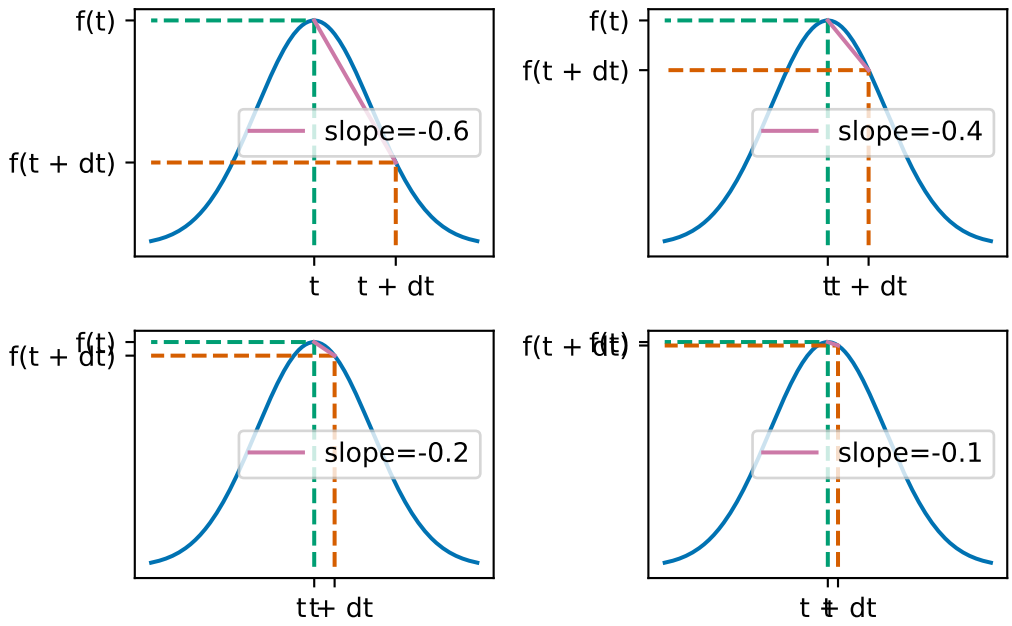


Figure 2.7: Sample straight-line approximations to the slope of a curve.

We can apply similar understanding no matter what the underlying function is. For example, we could do the same things with more complicated real world data. Here is an example using climate data (data source: Met Office).

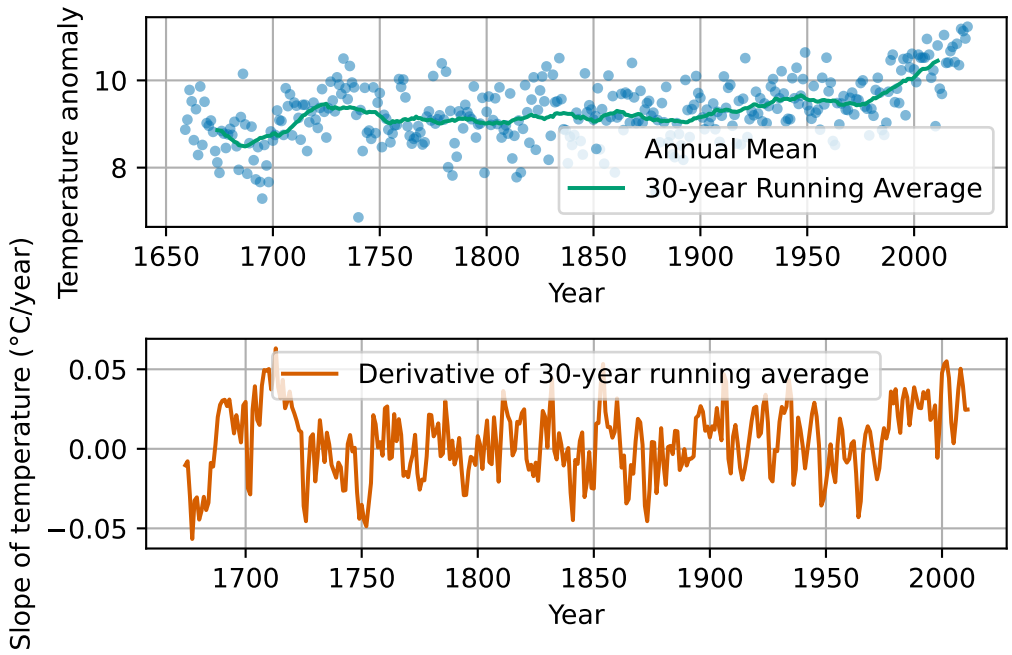


Figure 2.8: Central England Temperature anomaly. Annual Average 1659-2025.

2.3 Formulation in terms of formulae

Let's recall some of the more formal ideas that you learnt last year.

Definition 2.1. Let $f\colon (a,b) \to \mathbb{R}$ be a function on an interval (a,b) of the real line. We say that f is **differentiable** if for all $x \in (a,b)$ the limit:

$$\lim_{h\to 0} \frac{f(x+h)-f(x)}{h},$$

exists and is finite. If f is differentiable then we call the function $g\colon (a,b) \to \mathbb{R}$ the **derivative** of f defined by

$$g(x) = \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}.$$

We will write $f'(x) := g(x)$ or $\frac{\mathrm{d}}{\mathrm{d}x} f(x) = g(x)$.

Last year, you learnt some formula for derivatives. First that you can manipulate derivatives

in nice ways:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}x}(af_1(x)+bf_2(x)) &= a\frac{\mathrm{d}}{\mathrm{d}x}f_1(x)+b\frac{\mathrm{d}}{\mathrm{d}x}f_2(x) && \text{(sum rule)}\\
\frac{\mathrm{d}}{\mathrm{d}x}(f_1(x)f_2(x)) &= \left(\frac{\mathrm{d}}{\mathrm{d}x}f_1(x)\right)f_2(x)+f_1(x)\left(\frac{\mathrm{d}}{\mathrm{d}x}f_2(x)\right) && \text{(product rule)}\\
\frac{\mathrm{d}}{\mathrm{d}x}(f_1(f_2(x))) &= \frac{\mathrm{d}}{\mathrm{d}y}f_1(y)\Big|_{y=f_2(x)}\frac{\mathrm{d}}{\mathrm{d}x}f_2(x) && \text{(chain rule).}
\end{aligned}$$

You also learnt some derivatives of special functions:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}x}(x^p) &= px^{p-1}\\
\frac{\mathrm{d}}{\mathrm{d}x}\sin(x) &= \cos(x)\\
\frac{\mathrm{d}}{\mathrm{d}x}\cos(x) &= -\sin(x)\\
\frac{\mathrm{d}}{\mathrm{d}x}\exp(x) &= \exp(x).
\end{aligned}$$

Exercise 2.1. Compute the derivatives of the following functions:

1. $f_1(x) = 3x^2 - 5x + 1$
2. $f_2(x) = \sin(x^2)$
3. $f_3(x) = \exp(-x)\sin(2x)$
4. $f_4(x) = \frac{1}{\eta(x)}$ (for an arbitrary function η)
5. $f_5(x) = \tan(x)$. (*Hint:* use the fact that $\tan(x) = \sin(x)/\cos(x)$ then use the your answer for f_4 and the product and chain rules).

You also learnt about the opposite process to taking derivatives - integration - using Riemann sums:

Definition 2.2. Let $f\colon (a,b) \to \mathbb{R}$ be a function on the interval (a,b) of the real line.

Let $x_0 = a < x_1 < ... < x_n = b$ be a partition of the interval (a,b) into n smaller intervals (x_{i-1}, x_i) for $i = 1, \dots, n$. We choose a point c_i in each interval (x_{i-1}, x_i) and let $h = \max_i x_i - x_{i-1}$. We define the **Riemann sum** of f by

$$s_n = \sum_{i=1}^{n} f(c_i)(x_i - x_{i-1}).$$

Then we define the **integral** of f over (a,b) is given by:

$$\int_a^b f(x)\,\mathrm{d}x = \lim_{h\to 0}\sum_{i=1}^{n} f(c_i)(x_i - x_{i-1}).$$

You learnt the following properties of integration:

$$\int_a^b c_1 f_1(x) + c_2 f_2(x)\,\mathrm{d}x = c_1 \int_a^b f_1(x)\,\mathrm{d}x + c_2 \int_a^b f_2(x)\,\mathrm{d}x$$
$$\int_a^c f(x)\,\mathrm{d}x + \int_c^b f(x)\,\mathrm{d}x = \int_a^b f(x)\,\mathrm{d}x.$$

You learnt that using the fundamental theorem of calculus you can compute integrals by *inverting* the derivative process:

$$\int_a^b x^p\,\mathrm{d}x = \left[\frac{1}{p+1}x^{p+1}\right]_{x=a}^{b} = \frac{1}{p+1}(b^{p+1} - a^{p+1}).$$

Exercise 2.2. Find the integrals of the following functions over $(0, 1)$.

1. $f_1(x) = 3x^2 - 5x + 1$.
2. $f_2(x) = x + \frac{1}{x^2}$.

Remark 2.1. The problem of finding distance over the time interval $(0, 1)$, given speed, $S(t)$, given in (2.1), can be solved using integration.

Indeed, using that $D'(t) = S(t)$, the fundamental theorem of calculus tells us that $D(t)$ is the integral of $S(t)$ with respect to time:

$$\begin{aligned}
D(1) &= \int_0^1 S(t)\,\mathrm{d}t = \int_0^1 -(t-1)(t+0.5)\,\mathrm{d}t \\
&= \int_0^1 -t^2 + 0.5t + 0.5\,\mathrm{d}t \\
&= \left[-\frac{1}{3}t^3 + \frac{0.5}{2}t^2 + 0.5t\right]_{t=0}^{1} \\
&= (-\frac{1}{3} \times 1^3 + 0.25 \times 1^2 + 0.5 \times 1) - (-\frac{1}{3} \times 0^3 + 0.25 \times 0^2 + 0.5 \times 0) \\
&= \frac{5}{12} \approx 0.416667.
\end{aligned}$$

The rest of our section work in this week's lectures is to solve similar equations in the case that our formula for $S(t)$ would depend on t and also D.

2.4 Differential equations

We have already seen and solved one differential equation - when we solved for the distance travel as a function of time: $D'(t) = S(t)$. That equation related the derivative of D to a function of time $S(t)$. When a model takes the form of an equation which involves one or more derivatives then it is called a **differential equation**.

Most models of dynamic processes take the form of differential equations including climate models, many models in engineering and science, financial models, physics engines in computer games and many more!

We are interested in equations of the form:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

The data (things given to us) are:

- the initial time t_0
- the initial value of y at the initial time y_0 - we call this the *initial condition*
- the right hand side function f which can be a function of both time and the solution y,

and we must find a function $y\colon [0, T) \to \mathbb{R}$ for some final time T.

We have seen one example above with $f(t, y) = S(t) = -(t - 1)(t + 0.5)$, $t_0 = 0$ and $y_0 = 0$. We will see more later.

Remark 2.2. One important consideration is that while many clever ways to solve differential equations exactly exist, most differential equations cannot be solved by hand. Thus numerical methods are not only convenient ways to find solutions of differential equations, in many cases they are essential.

The downside of a computational approach arises now since we are *approximating* the true solution. We will cover more on this in the next lecture.

Example 2.1 (Example differential equation - an object in free fall)**.** Consider the following simple model for an object falling from a large height, based on the two assumptions:

1. all objects are attracted downward with an acceleration due to gravity of $9.81\,\mathrm{m/s^2}$;
2. air resistance causes an object to decelerate in proportion to its speed (i.e. the faster it travels the greater the air resistance).

The *net acceleration downwards* (i.e. rate of change of speed, $S'(t)$) is given by the sum of the two forces g and $-kS(t)$:

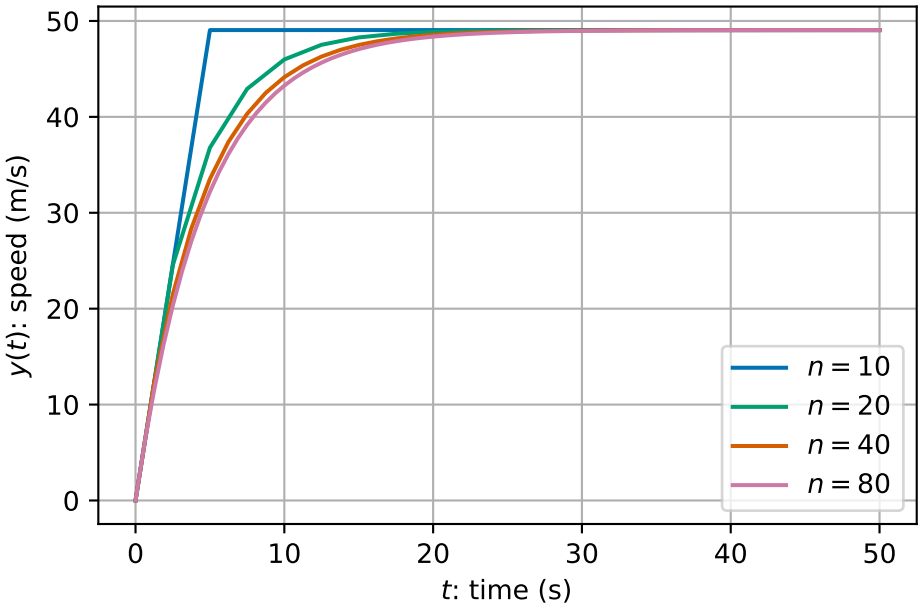
$$S'(t) = g - kS(t).$$

How can we solve this equation?

- We cannot solve this integral exactly:

$$S(t) = S(0) + \int_0^T g - kS(t)\,\mathrm{d}t.$$

- There are techniques to solve this problem analytically (using so-called separation of variables) – but we are simple computer scientists looking for simple computer-based solutions instead!
- We can do similar to the above idea where we divide the time period into lots of small intervals and assume that everything is approximately constant on each time interval…



We see that as we take more time steps, we converge to a smooth solution.

2.5 Euler's method

The approach used in Example 2.1 can be applied to any differential equation in the generic format:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

The general algorithm is very simple:

1. Set initial values $t^{(0)}$ and $y^{(0)}$ from the initial condition.

2. Loop over all time steps, until the final time T, updating using the formula:

$$y^{(i+1)} = y^{(i)} + hf(t^{(i)}, y^{(i)})$$
$$t^{(i+1)} = t^{(i)} + h.$$

The algorithm has one free parameter: the number of time steps, n. Choosing more time steps increases the cost of the algorithm helps us converge to the solution (more on this later). Once the number of time steps is fixed, we can calculate the time step $h = (T - t_0)/(n - 1)$.

Example 2.2. Take two steps of Euler's method to approximate the solution of

$$y'(t) = -(y(t))^2 + 3t^2 \quad \text{subject to the initial condition} \quad y(1) = 2,$$

for $1 \leq t \leq 2$.

In this example we have:

- $n = 2$
- $t_0 = 1$
- $y_0 = 2$
- $T = 2$
- $h = (2 - 1)/2 = 0.5$
- $f(t, y) = -y^2 + 3t^2$.

We start with $t^{(0)} = 1$ and $y^{(0)} = 2$.

- Step 1:

$$\begin{aligned} y^{(1)} &= y^{(0)} + hf(t^{(0)}, y^{(0)}) \\ &= 2 + 0.5(-(2)^2 + 3 \times 1^2) = 1.5 \\ t^{(1)} &= t^{(0)} + h \\ &= 1 + 0.5 = 1.5 \end{aligned}$$

- Step 2:

$$\begin{aligned} y^{(2)} &= y^{(1)} + hf(t^{(1)}, y^{(1)}) \\ &= 1.5 + 0.5(-(1.5)^2 + 3 \times 1.5^2) = 3.75 \\ t^{(2)} &= t^{(1)} + h \\ &= 1.5 + 0.5 = 2.0 \end{aligned}$$

Exercise 2.3. Complete the next two time steps from Example 2.2 to find an approximation of the solution at $T = 3$.

We see that Euler's method is a very general method that is very easy to apply. We can very quickly apply lots of very small time steps for (what we hope will be) a good approximation of the solution. This generality and simplicity is very powerful.

In the next lecture, we will measure the error of this approach and see how we can use our understanding of the errors to find an improve formulation.

2.6 Summary

This lecture introduces you to how we study dynamic problems, situations where things change over time, and why numerical methods are such powerful tools for computer scientists. You have explored familiar ideas like speed and distance, seeing how they connect through derivatives and integrals. From there, the lecture builds up your understanding of what a derivative really means (both as a limit and as a slope), how to compute them, and how integration reverses the process.

Once these foundations are in place, you move into the world of differential equations, which are the backbone of models in physics, engineering, climate science, and even games. Because most real-world equations can't be solved exactly, the lecture shows you how to approximate solutions using Euler's method: a simple, step-by-step numerical technique that computers can apply easily. By the end, you've seen how to turn a continuous, time-dependent problem into something you can compute, setting you up for deeper work on accuracy and error analysis in the next session.

3 Analysis of errors and systems of equations

3.1 Exact derivatives and exact solutions

In a small number of cases, it is possible to solve differential equations *exactly*. That is to write down a simple expression for the function which satisfies both the differential equation and its initial condition. This is rare and is the main reason why we use numerical methods. However, the special cases where we can find exact solutions are useful for testing our numerical methods.

Example 3.1. Consider the function $y(t) = t^3$.

We can find the derivative of $y(t)$:

$$\begin{aligned} y'(t) &= \lim_{h\to 0} \frac{y(t+h) - y(t)}{h} \\ &= \lim_{h\to 0} \frac{(t+h)^3 - t^3}{h} \\ &= \lim_{h\to 0} \frac{t^3 + 3t^2h + 3th^2 + h^3 - t^3}{h} \\ &= \lim_{h\to 0} \frac{3t^2h + 3th^2 + h^3}{h} \\ &= \lim_{h\to 0} 3t^2 + 3th + h^2 \\ &= 3t^2. \end{aligned}$$

By working backwards, we can make up our own differential equation that has $y(t)$ as a known solution. When $y(t) = t^3$, we can compute that

$$y'(t) = 3t^2 = \frac{3y(t)}{t}.$$

Hence, we know the solution to the following equation:

$$y'(t) = \frac{3y(t)}{t} \quad \text{subject to} \quad y(1) = 1. \tag{3.1}$$

If we solve this differential equation for t between 1 and 2, say, then we know that the exact answer when $t = 2$ is $y(2) = 8$.

Note here that we start from $t = 1$ to avoid $t = 0$.

We will use this differential equation where we know the exact solution to test Euler's method. Recall that for a differential equation given by:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

Euler's method is given by:

1. Set initial values $t^{(0)}$ and $y^{(0)}$ from the initial condition.
2. Loop over all time steps, until the final time T, updating using the formula:

$$y^{(i+1)} = y^{(i)} + hf(t^{(i)}, y^{(i)})$$
$$t^{(i+1)} = t^{(i)} + h.$$

Exercise 3.1. Compute two time steps of Euler's method to approximate the solution of (3.1) at time $T = 2$. What is the error between your computed solution and the exact solution?

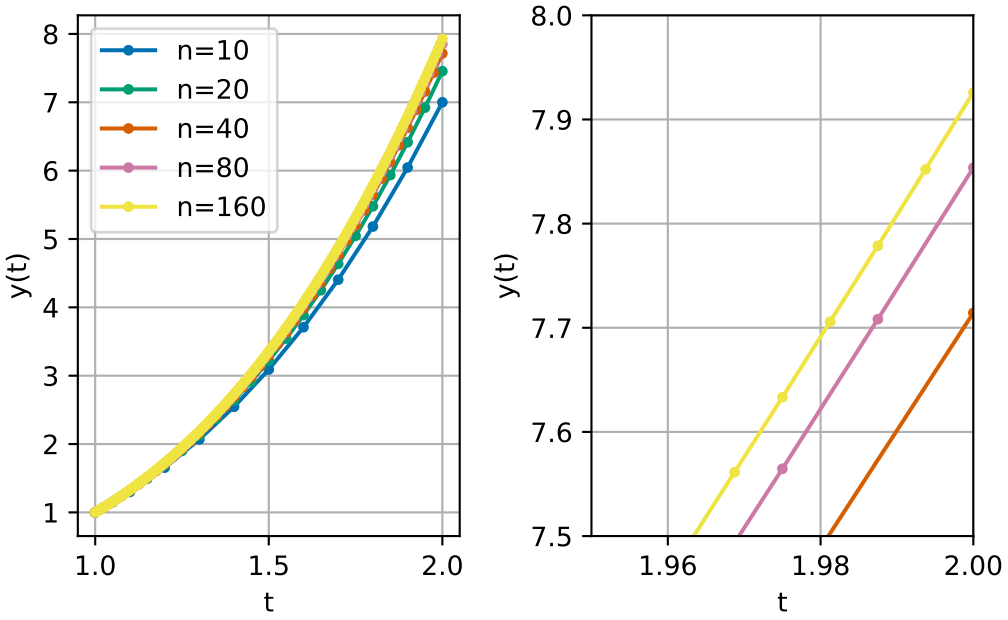
We can also implement Euler's method in code:

```
data = {}

for n in [10, 20, 40, 80, 160]:
    # set up storage for all time steps
    h = (2 - 1) / n
    t, y = np.empty((n + 1,)), np.empty((n + 1,))

    # perform time stepping
    t[0], y[0] = 1.0, 1.0
    for i in range(n):
        t[i + 1], y[i + 1] = t[i] + h, y[i] + h * (3 * y[i]) / t[i]

    # store data
    data[n] = (t, y)
```



We can see that as we increase the number of steps n (reduce the time step h), our solutions become better and better. We can also compute these values using the *absolute error*

$$\text{absolute error} = |y^{(n)} - y(2)|,$$

where $y^{(n)}$ is the solution at the final time step.

Table 3.1: A convergence study for Euler's method

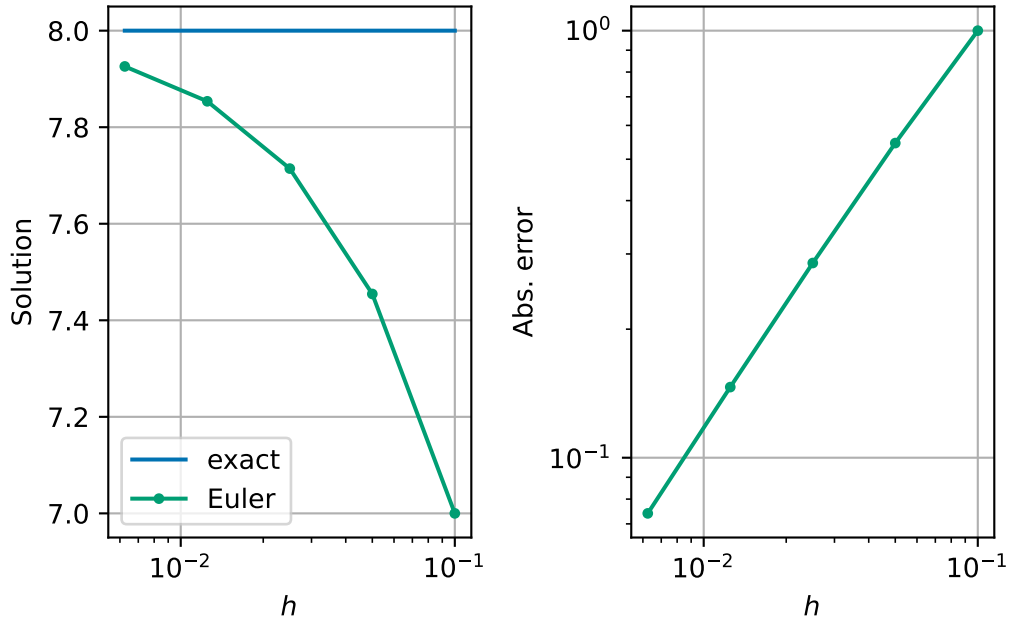
Table 3.1

	n	h	Solution	Abs. error	ratio
0	10	\(1.00000\times 10^{-1}\)	7.000000	\(1.00000\times 10^{0}\)	---
1	20	\(5.00000\times 10^{-2}\)	7.454545	\(5.45455\times 10^{-1}\)	0.545455
2	40	\(2.50000\times 10^{-2}\)	7.714286	\(2.85714\times 10^{-1}\)	0.523810
3	80	\(1.25000\times 10^{-2}\)	7.853659	\(1.46341\times 10^{-1}\)	0.512195
4	160	\(6.25000\times 10^{-3}\)	7.925926	\(7.40741\times 10^{-2}\)	0.506173

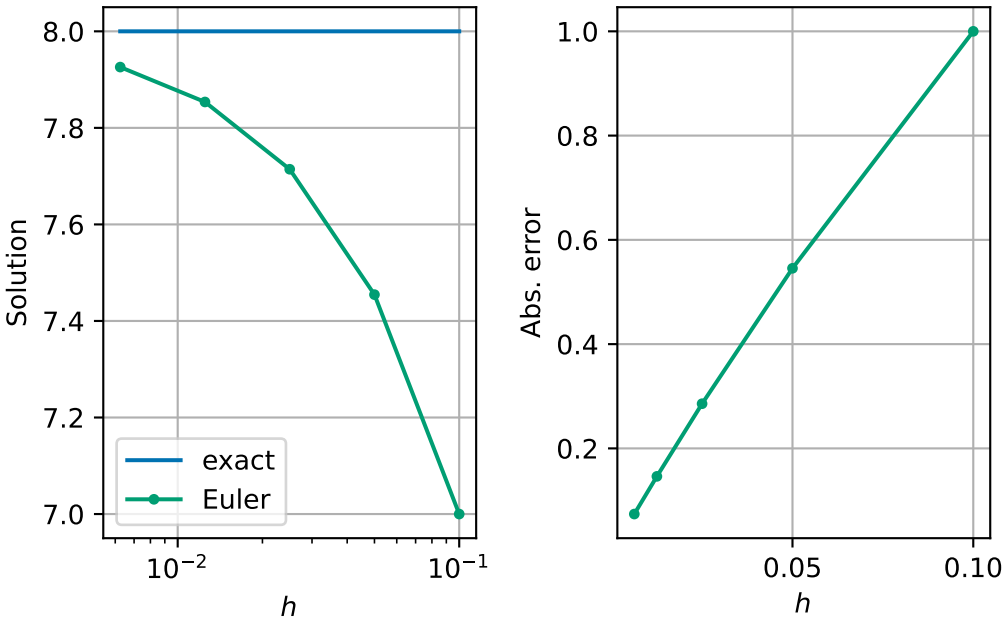
This table shows that indeed the absolute error is decreasing as $h \to 0$. We also compute the ratio between different errors. We see that each time h is halved, that the error is approximately halved. In other words, the error is approximately proportional to h.

Exercise 3.2. What might we expect the computed solution to be if we halved h one more time?

Something interesting happens when we plot h against the absolute error for Euler's method - we see that the errors all fall on a straight line. Perhaps this is not too strange since we think that the error is approximately proportional to step size, so this line should be straight either on a linear or log scale.



Remark 3.1. Using log scales is really important when considering variables like h or error. Contrast the above nice plot, with nice evenly spread data points, with these linear scaled plots:



Exercise 3.3. Why don't we use a log scale for solution too?

Remark 3.2. It is important to note that in this lecture, we are measuring the error at the final time point as a measure of how good our method is. This is a *global error* that measures the accumulation of *local errors* which happen at each time step. Other possible global errors include finding the maximum or average error over all time steps.

Sometimes we also care more about other, more physical based properties of our solution. For example, in long-time simulations scientists often want to ensure that the laws of thermodynamics are implemented exactly – i.e. that energy is conserved and that entropy increases.

3.2 Big O notation

TODO examples from the rest of the module. check this bit with Haiko

When considering algorithm complexity, you have already seen big O notation. For example:

- Gaussian elimination requires $O(n^3)$ operations when n is large,
- Backward substitution requires $O(n^2)$ operations when n is large.

For large values of n the *highest* powers of n are the *most* significant. For smaller values of h, it is the *lowest* powers of h that are the most significant. When $h = 0.001$, h is much bigger than h^2, for example.

We can make use of the big O notation in either case. Formally, we say that $f(h) = O(h^p)$ as $h \to 0$ if there exists a constant $C > 0$ such that $|f(h)| < C|h|^p$ for all small enough h.

Example 3.2. Let $f(x) = 2x^2 + 4x^3 + x^5 + 2x^6$,

- then $f(x) = O(x^6)$ as $x \to \infty$,
- and $f(x) = O(x^2)$ as $x \to 0$.

In this notation, we can say that **the error in Euler's method is $O(h)$.**

3.3 The midpoint method – an improvement in Euler's method

We want to derive a new improved method for solving differential equations. We aim to produce a method with a smaller error for a similar amount of work per time step.

Let's assume that the error in Euler's method is exactly proportional to h. In this case, if we halve h, we halve the error. In this derivation, we will start from the initial condition and want to approximate the solution after a time h. We will label the exact solution at $t = h$ by y_{exact}.

- Then, we continue by computing one time step from $y^{(0)}$ with time step h - we call this solution z_1. We label error between the exact solution and z_1 by E_1.
- We then take two time steps from $y^{(0)}$ now with time step $h/2$ - we call this solution z_2. We label the error between the exact solution and z_2 by E_2. Since in this derivation, the error is exactly proportional to the time step, we can see that $E_2 = \frac{E_1}{2}$.

In equations, we can express these relations as:

$$z_1 - y_{\text{exact}} = E_1$$
$$z_2 - y_{\text{exact}} = E_2 = \frac{E_1}{2}.$$

Subtracting twice the second equation from the first equation gives:

$$y_{\text{exact}} = 2z_2 - z_1.$$

If our assumption that the error in Euler's method is exactly proportional to h, combining z_1 and z_2 would give a solution with zero error! However, our assumption is in fact approximately true as we found in Table 3.1. So we might hope that we have found a method with smaller error than Euler's method with three times the cost of Euler's method (i.e. 3 Euler steps per one time step of this method - one to find z_1 and two to find z_2).

We can even formulate this method more compactly. Writing our new scheme in full, for a general time step i, we have for the one step approach:

$$z_1 = y^{(i)} + hf(t^{(i)}, y^{(i)}),$$

and for the two step approach:

$$\begin{aligned} k &= y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)}) \\ m &= t^{(i)} + \frac{h}{2} \\ z_2 &= k + \frac{h}{2} f(m, k). \end{aligned}$$

Combining z_1 and z_2 as suggested above we see:

$$\begin{aligned} y^{(i+1)} &= 2z_2 - z_1 \\ &= 2\left(k + \frac{h}{2} f(m,k)\right) - \left(y^{(i)} + hf(t^{(i)}, y^{(i)})\right) && \text{(substitute for } z_1 \text{ and } z_2) \\ &= 2k - y^{(i)} + h\left(f(m,k) - f(t^{(i)}, y^{(i)})\right) && \text{(rearrange)} \\ &= 2\left(y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)})\right) - y^{(i)} + h\left(f(m,k) - f(t^{(i)}, y^{(i)})\right) && \text{(substitute for } k) \\ &= 2y^{(i)} - y^{(i)} + h\left(f(t^{(i)}, y^{(i)}) + f(m,k) - f(t^{(i)}, y^{(i)})\right) \\ &= y^{(i)} + hf(m,k). \end{aligned}$$

The midpoint method for solving differential equations is then as follows:

1. Initialise starting values $t^{(0)}$ and $y^{(0)}$.
2. Loop over all time steps, until the final time, updating using the formulae:

$$\begin{aligned} k &= y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)}) \\ m &= t^{(i)} + \frac{h}{2} \\ y^{(i+1)} &= y^{(i)} + hf(m,k) \\ t^{(i+1)} &= t^{(i)} + h. \end{aligned}$$

Remark 3.3. In the end the cost of this formulation of the midpoint method is twice as large as Euler's method.

Example 3.3. Take two steps of the midpoint rule to approximate the solution of

$$y'(t) = -y(t)^2 \quad \text{subject to the initial condition} \quad y(0) = 2,$$

for $0 \leq t \leq 2$.

In this example we have:

- $n = 2$
- $t_0 = 0$
- $y_0 = 2$
- $T = 2$
- $h = (2 - 0)/2 = 1.0$
- $f(t, y) = -y^2$.

We start with $t^{(0)} = 0$ and $y^{(0)} = 2$.

- Step 1:

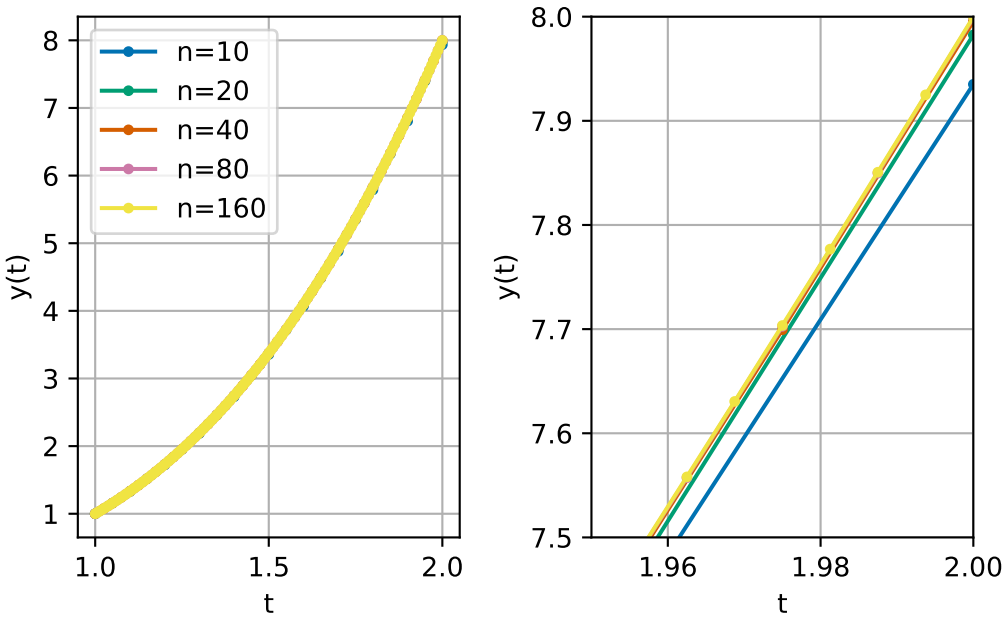
$$\begin{aligned} k &= y^{(0)} + \frac{h}{2} f(t^{(0)}, y^{(0)}) \\ &= 2 + \frac{1.0}{2} \times (-(2)^2) \\ &= 2 + 0.5 \times -4 = 0.0 \\ m &= t^{(0)} + \frac{h}{2} \\ &= 0 + \frac{1.0}{2} = 0.5 \\ y^{(1)} &= y^{(0)} + hf(m, k) \\ &= 2 + 1.0(-(0.0)^2) \\ &= 2 + 1.0 \times -0.0 = 2.0 \\ t^{(1)} &= t^{(0)} + h \\ &= 0 + 1.0 = 1.0. \end{aligned}$$

- Step 2:

$$\begin{aligned} k &= y^{(1)} + \frac{h}{2} f(t^{(1)}, y^{(1)}) \\ &= 2.0 + \frac{1.0}{2} \times (-(2.0)^2) \\ &= 2.0 + 0.5 \times -4.0 = 0.0 \\ m &= t^{(1)} + \frac{h}{2} \\ &= 1.0 + \frac{1.0}{2} = 1.5 \\ y^{(2)} &= y^{(1)} + hf(m, k) \\ &= 2.0 + 1.0(-(0.0)^2) \\ &= 2.0 + 1.0 \times -0.0 = 2.0 \\ t^{(2)} &= t^{(1)} + h \\ &= 1.0 + 1.0 = 2.0. \end{aligned}$$

3.3.1 Numerical results

We can also implement the method in code and explore how the error changes as we change the number of steps. For this test we use the same test as we did for Euler's method (3.1).



We already see that we are much closer to the solution.

Let's compute the errors:

Table 3.2: A convergence study for the midpoint method

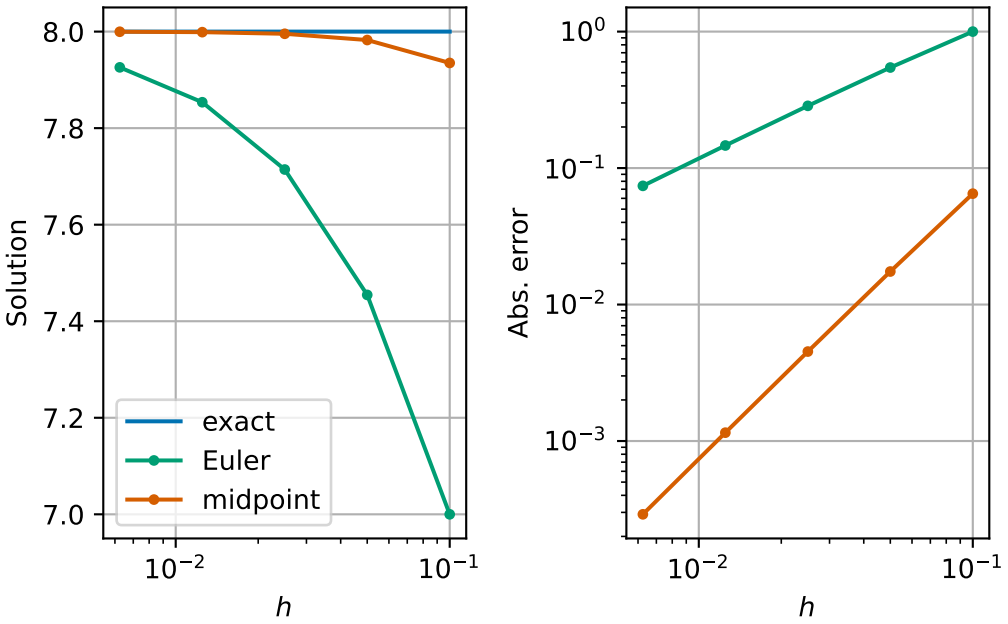
Table 3.2

	n	h	Solution	Abs. error	ratio
0	10	\(1.00000\times 10^{-1}\)	7.935060	\(6.49403\times 10^{-2}\)	---
1	20	\(5.00000\times 10^{-2}\)	7.982538	\(1.74619\times 10^{-2}\)	0.268892
2	40	\(2.50000\times 10^{-2}\)	7.995475	\(4.52485\times 10^{-3}\)	0.259127
3	80	\(1.25000\times 10^{-2}\)	7.998849	\(1.15145\times 10^{-3}\)	0.254473
4	160	\(6.25000\times 10^{-3}\)	7.999710	\(2.90410\times 10^{-4}\)	0.252212

We now see that the error reduces by a *quarter* each time we reduce h by half. This means that the error is approximately proportional to h^2, or equivalently, we can say that the error is $O(h^2)$ as $h \to 0$.

This is a significant improvement on Euler's method. We say that the midpoint method is **second order**, whereas Euler's method is just **first order**.

We can see the significant reduction in error by plotting the errors for both methods:



Our interesting phenomenon of the straight line on the log-log plot of h and absolute error has happened again for the midpoint method. The difference now is that the line is much steeper. In a log-log plot, straight lines correspond to power laws (i.e., Ch^p) and the slope indicates the exponent in the power law (p). Indeed if $\text{error}(h) \approx Ch^p$ then

$$\log(\text{error}(h)) \approx \log C + p \log h.$$

We are seeing that our method behaves as if the error is proportional to h^2.

3.4 Balancing cost and accuracy

At first glance, it may seem that we now have a trade-off between reducing the error but at a higher cost by using the midpoint method. We can test this more explicitly. We repeat the experiments but use two additional measures of cost: the run time and the number of right-hand side (f) evaluations. When working with real-world systems, evaluating the right-hand side function often forms the main computational cost of both Euler's method and the midpoint method.

Table 3.3: A convergence study for Euler's method with running costs

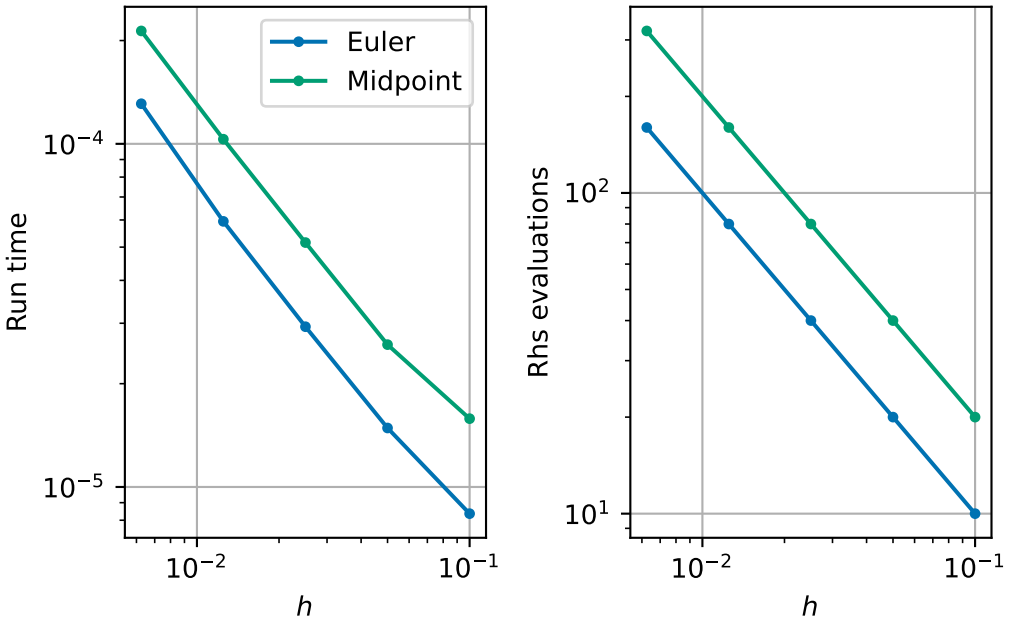
Table 3.3

	n	h	Solution	Abs. error	Run time	Rh
0	10	\(1.00000\times 10^{-1}\)	7.000000	\(1.00000\times 10^{0}\)	\(8.36060\times 10^{-6}\)	10
1	20	\(5.00000\times 10^{-2}\)	7.454545	\(5.45455\times 10^{-1}\)	\(1.48558\times 10^{-5}\)	20
2	40	\(2.50000\times 10^{-2}\)	7.714286	\(2.85714\times 10^{-1}\)	\(2.93118\times 10^{-5}\)	40
3	80	\(1.25000\times 10^{-2}\)	7.853659	\(1.46341\times 10^{-1}\)	\(5.94322\times 10^{-5}\)	80
4	160	\(6.25000\times 10^{-3}\)	7.925926	\(7.40741\times 10^{-2}\)	\(1.30877\times 10^{-4}\)	160

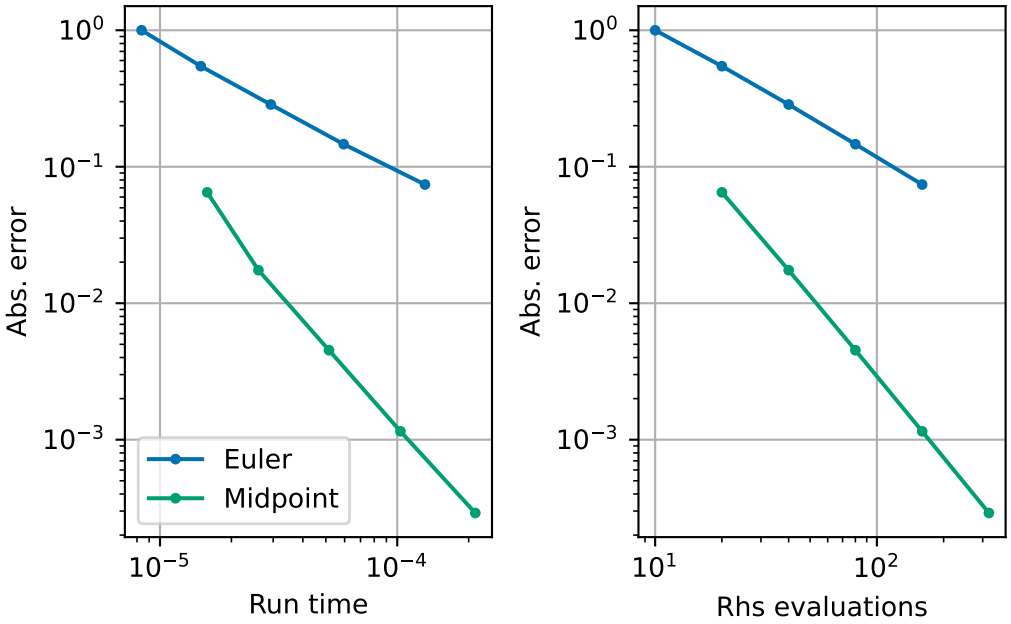
Table 3.4: A convergence study for Midpoint's method with running costs

Table 3.4

	n	h	Solution	Abs. error	Run time	Rh
0	10	\(1.00000\times 10^{-1}\)	7.935060	\(6.49403\times 10^{-2}\)	\(1.58156\times 10^{-5}\)	20
1	20	\(5.00000\times 10^{-2}\)	7.982538	\(1.74619\times 10^{-2}\)	\(2.59906\times 10^{-5}\)	40
2	40	\(2.50000\times 10^{-2}\)	7.995475	\(4.52485\times 10^{-3}\)	\(5.15565\times 10^{-5}\)	80
3	80	\(1.25000\times 10^{-2}\)	7.998849	\(1.15145\times 10^{-3}\)	\(1.03160\times 10^{-4}\)	160
4	160	\(6.25000\times 10^{-3}\)	7.999710	\(2.90410\times 10^{-4}\)	\(2.13314\times 10^{-4}\)	320



However, we can make this plot another way by plotting the cost against the error.



We see now that whichever way you want to measure cost, you see that for a fixed cost the midpoint method gives a lower error. Although the midpoint method costs about twice as much per time step, its higher order accuracy means we need far fewer steps to reach a desired accuracy. So the best way to think is that if you have a budget of, say, right-hand side evaluations, N, you can afford for your computations, then in general, you should choose to use the midpoint method with $n = N/2$ time steps in order to minimise the error.

4 Summary

We have found different ways to computationally measure the quality of an algorithm to solve differential equations. We have used one way of measuring errors to derive an improvement to Euler's method called the midpoint method. We see that Euler's method is a first order accurate method and the midpoint method is second order accurate. The computational cost of the midpoint method is twice as high as Euler's method, but the higher order accuracy outweighs the higher computational cost in general.